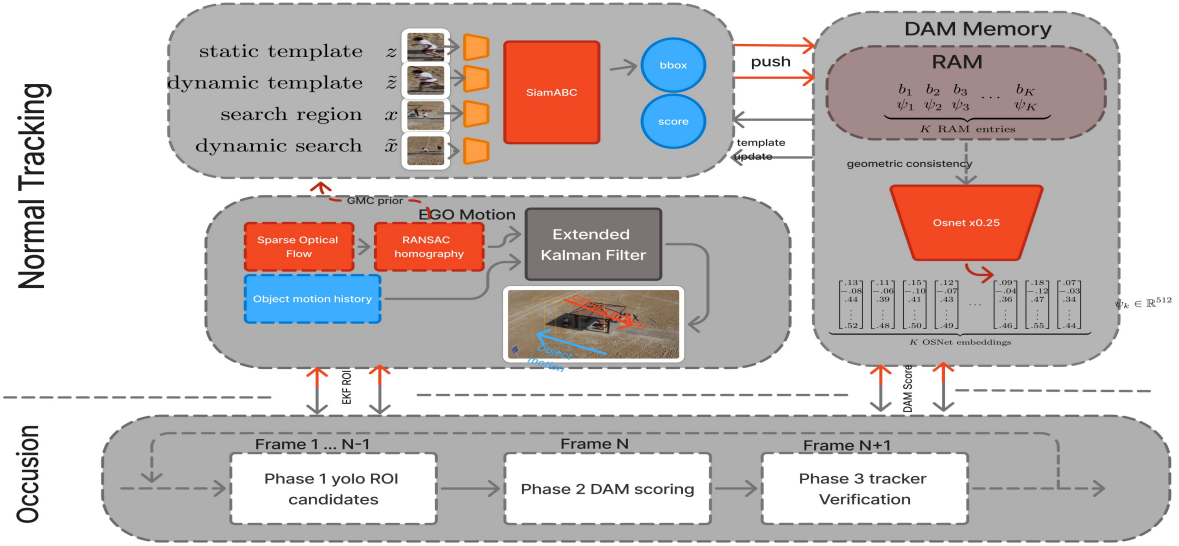


SiamRAM: Confidence-Gated Siamese Tracking with Distractor-Aware Memory and Multi-Phase Reacquisition for Embedded UAV Tracking



1 Introduction

Tracking a single user-specified object across a UAV video stream is a core capability for aerial monitoring, yet it is repeatedly broken by three failures. Under *occlusion* the target hides and the box latches onto whatever is left; under *distractors* a look-alike object steals the box; and under *ego-motion* a camera pan slides the whole scene, so a motion model that assumes the *object* moved aims the search window at empty sky.

Existing trackers trade these failures against speed. Lightweight Siamese matchers [1, 2, 3] are fast enough for an embedded GPU but, lacking memory or re-detection, drift permanently once any of the three failures occurs. Memory- and re-detection-based trackers are sturdier but spend that robustness every frame, which is too slow for a Jetson-class device. We argue the two need not conflict: the heavy machinery is only needed in the rare frames where the tracker is actually failing, and the tracker already emits a signal — its classification score $\hat{c} \in [0, 1]$ — that could say when. The obstacle is that the off-the-shelf $\hat{c}$ is not trustworthy: trained only on positive pairs, it saturates to ≈ 1 on background and distractors alike, so a single threshold on it is meaningless.

SiamRAM turns that signal into a usable gate and builds on top of it. We first recalibrate $\hat{c}$ so that one threshold separates “target present” from “target absent,” then attach three robustness modules that the recalibrated $\hat{c}$ switches on only when needed. The result keeps a fast core’s per-frame cost in the common case while recovering a memory tracker’s robustness in the hard case, inside an embedded real-time budget.

Contributions. Each contribution is a testable design claim; the supporting evidence in this paper is mechanistic and efficiency-based, and the accuracy evaluation we still owe is scoped in §8.

- **Confidence recalibration** (§4): we identify a *confidence collapse* in the pre-trained core and correct it with hard-negative fine-tuning, so that $\hat{c}$ becomes a usable gate rather than a saturated constant.
- **Confidence-gated escalation** (§3–7): every heavy module reads the *same* scalar $\hat{c}$, so compute follows difficulty; we quantify the resulting per-frame budget in §7.
- **Camera-aware filtering and a two-tier identity mem-**

ory (§5–6): we subtract estimated camera motion *inside* the Kalman predict step so the velocity estimate stays object-only, and we admit a recovery anchor only after it proves temporally stable, so transient views never seed recovery.

- **Self-calibrating multi-phase reacquisition** (§7): recovery decouples *where* (EKF), *which* (identity), and *confirm* (the tracker itself), and its one acceptance threshold adapts online instead of being tuned per video.

2 Related Work

Efficient Siamese tracking. Siamese matchers cast tracking as template–search correlation [1, 2]; SiamABC [3] adds dynamic templates and test-time batch calibration [5] for efficiency on edge devices. These cores are our starting point but have no mechanism to recover once the box is lost. **Long-term tracking** adds re-detection to reset after loss; we follow that idea but gate it on a recalibrated confidence so it costs nothing in the common case, and verify re-detections with the tracker itself rather than trusting the detector. **Re-identification** embeddings such as OSNet [4] map an identity to a stable vector; we use them not per frame but as a curated, temporally filtered memory for distractor-resistant recovery. **Motion and camera compensation** via Lucas–Kanade flow [8] and Kalman filtering [9] are classical; our contribution is to fold the estimated homography *into* the filter’s predict step so that ego-motion is removed from the object-velocity state rather than merely from the search window.

3 Normal Tracking: the SiamABC Core

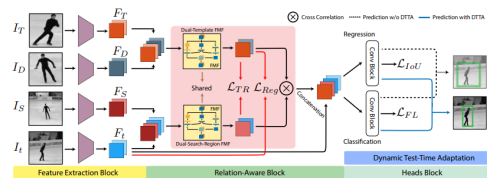


Figure 1: The core processes four crops per frame: frozen static template z , evolving dynamic template $\tilde{z}$, search region x , and dynamic search $\tilde{x}$. A shared backbone encodes all four; cross-attention fuses each template–search pair; the head reads out a box and a confidence $\hat{c}$.

The core is the **M** variant of SiamABC [3]: a ResNet-style backbone kept to four stages, depthwise-separable convolutions in the head, and no IoU branch. Templates are cropped at

128×128 px and search regions at 256×256 px; the head decodes a 16×16 grid at stride 16.

From pixels to a box and a score. Each frame uses four crops (Fig. 1): the static template z from frame 0, a dynamic template $\tilde{z}$ refreshed as the target’s appearance evolves, the current search crop x , and a dynamic search $\tilde{x}$. A shared backbone E and neck N embed all four,

$$f_{\bullet} = N(E(\bullet)), \quad \bullet \in \{z, \tilde{z}, x, \tilde{x}\}. \quad (1)$$

Cross-attention A and a second neck N' fuse each template–search pair,

$$m_z = N'(A([f_z; f_{\tilde{z}}])), \quad m_x = N'(A([f_x; f_{\tilde{x}}])), \quad (2)$$

and the BoxTower head T correlates them into a per-cell box offset and a confidence:

$$(\hat{\Delta}, \hat{c}) = T(m_x, m_z), \quad \hat{c} = \sigma(\ell) \in [0, 1]. \quad (3)$$

Boxes are decoded FCOS-style [6]: cell (i, j) at image location g_{ij} predicts four edge distances $(\hat{l}, \hat{t}, \hat{r}, \hat{b})$, giving $[g_x - \hat{l}, g_y - \hat{t}, g_x + \hat{r}, g_y + \hat{b}]$, and we keep the cell of largest $\hat{c}$. That scalar $\hat{c}$ is the only signal the rest of SiamRAM reads.

Dynamic template, guarded against poisoning. A single frame-0 template goes stale as the target rotates, scales, and re-lights. We keep a rolling buffer of recent good frames (capacity $M=30$, shrunk to $M_w=8$ during a 5-frame warm-up) and periodically promote one to be the new $\tilde{z}$. To prevent a distractor frame from poisoning the template, a frame (I_k, b_k) with score s_k is admitted only if it is both confident and geometrically consistent with the previous box:

$$(s_k > \hat{\rho}_k \text{ or } s_k \geq \tau_u) \text{ and } \text{IoU}(b_k, b_{k-1}) \geq \tau_{\text{iou}}, \quad (4)$$

with $\tau_u=0.83$, $\tau_{\text{iou}}=0.60$, and $\hat{\rho}_k = \max(\rho_{\min}, (1-\lambda)\hat{\rho}_{k-1} + \lambda s_k)$ a floored score EMA ($\lambda=0.1$, $\rho_{\min}=0.50$). Every $N=40$ frames the newest admitted frame above τ_u becomes $\tilde{z}$; the IoU gate blocks a one-frame distractor jump from entering the template.

Test-time batch normalisation. Selected head BatchNorm layers adapt to the current crop rather than frozen training statistics [5], blending the running mean with the crop mean,

$$\mu_{\text{eff}} = \lambda \mu_{\text{batch}} + (1-\lambda) \mu_{\text{run}}, \quad \lambda=0.1, \quad (5)$$

(likewise for the variance) with affine weights frozen — cheap robustness to illumination drift, contingent on a calibrated $\hat{c}$ (§4).

4 Confidence Recalibration

The pre-trained SiamABC checkpoint is trained almost entirely on *positive* template–search pairs, where the target is always present in the search crop. Such training admits a degenerate shortcut — always predict a high score — at negligible loss. We call the result **confidence collapse**: $\hat{c}$ saturates to ≈ 1 on background, distractors, and occluded frames alike. Since every SiamRAM gate (template admission (4), occlusion entry, recovery acceptance) thresholds $\hat{c}$, an uninformative $\hat{c}$ makes all of them fire on noise. Recalibration is therefore a prerequisite, not a refinement.

We fine-tune the head and attention necks (backbone frozen) with negatives mixed in at $\approx 1:1$. A negative crop has the target provably absent, so its label map is all-zero ($y_{ij}=0$), and is produced two ways behind a hard zero-overlap check: *directional shifts* (the majority) push the crop centre $\geq 3.1d$ off the target (d the object diagonal), yielding hard negatives that

resemble a real search crop; *distractor crops* sample a same-size box elsewhere in the frame or from another sequence. With focal classification loss [7] and $p_{ij} = \hat{c}_{ij}$ over the grid Ω ,

$$\mathcal{L}_{\text{cls}} = \frac{1}{|\Omega|} \sum_{(i,j) \in \Omega} \text{FL}(p_{ij}, y_{ij}), \quad \text{FL}(p, 0) = -p^\gamma \log(1-p), \quad (6)$$

a pure-negative crop reduces to the right-hand term, minimised only as $p_{ij} \rightarrow 0$. The objective thus *penalises* a confident score when the target is absent — exactly the case positive-only training never showed — opening the separation that the downstream thresholds (e.g. 0.50 for occlusion, 0.83 for template admission) assume. We report the system with this recalibration in place; a quantitative calibration analysis is part of the planned evaluation (§8).

5 Moving-Camera Robustness

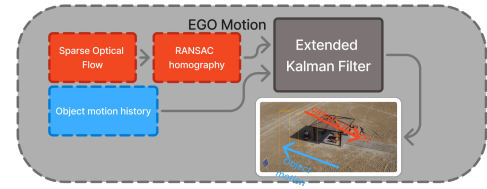


Figure 2: Sparse optical flow on masked-background points $\rightarrow$ RANSAC homography $H_t \rightarrow$ a warped-forward search prior and a camera-aware EKF that subtracts H_t before adding object velocity.

A drone pan shifts the target by tens of pixels per frame even when it is static relative to the ground. A constant-velocity Kalman filter mistakes that shift for object velocity, over-predicts, and on a hard pan drops $\hat{c}$ enough to declare a false occlusion. We measure the camera motion and subtract it.

Estimating camera motion. With the target region masked, we track background corners between frames by sparse Lucas–Kanade flow [8] and fit a 3×3 homography H_t by RANSAC on a downsampled grey image (long edge 300 px). A fast affine fit (“classic”) and a full projective fit on GFTT corners (“accurate”) are available. We accept H_t only when its inlier ratio exceeds 0.5 *and* it is geometrically plausible (scale $\in [0.70, 1.40]$, rotation $\leq 25^\circ$, corner displacement $\leq 25\%$ of frame width); otherwise we treat the frame as having no camera motion, since a wild homography is worse than none.

Folding it into the filter. Let $\pi(H, \cdot)$ warp a point through H . When H_t is accepted we (i) seed the search region at the warped-forward box $\hat{i}_t^{\text{GMC}} = \pi(H_t, b_{t-1})$, and (ii) advance a 4-D EKF with state $[c_x, c_y, v_x, v_y]^\top$ whose predict step removes the camera term before adding object velocity,

$$[c_x, c_y]_t \leftarrow \pi(H_t, [c_x, c_y]_{t-1}) + [v_x, v_y]_{t-1}, \quad P \leftarrow FPF^\top + Q. \quad (7)$$

Because the homography absorbs the scene slide, $[v_x, v_y]$ remains an estimate of *object-only* motion — which is what we rely on during occlusion, when there is no measurement. The EKF then runs predict-only with growing covariance $P \leftarrow P+Q$, anchoring the recovery search where the target should have drifted.

6 Distractor-Aware Appearance Memory

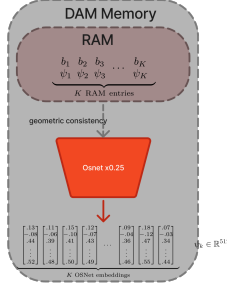


Figure 3: A geometry-gated short-term buffer (RAM) feeds OSNet embeddings to a temporally curated long-term bank (DRM). Only appearances that stay consistent over time become recovery anchors.

Geometry alone cannot separate the target from a look-alike beside it: both occupy plausible positions. We need *identity*, and a colour/texture histogram does not provide it — it changes as much when the target turns as when a different object appears. We instead embed each patch P with **OSNet-x0.25** [4], a re-ID network that maps one identity to nearby vectors across pose, and L2-normalise so cosine similarity is a dot product:

$$\psi = g(P)/(\|g(P)\| + \epsilon) \in \mathbb{R}^{512}, \quad g = \text{OSNet-x0.25}. \quad (8)$$

OSNet-x0.25 is small (≈ 0.20 M params, ≈ 0.16 GFLOPs/crop), which matters for the budget in §7.

Two tiers. A short-term **RAM** (capacity C_R) admits the current box only when it is geometrically consistent with the prior estimate ($\text{IoU}(b, \hat{b}_{t-1}) \geq \tau_{\text{ram}}=0.40$, area change $\leq \tau_a=0.25$), blocking distractor-sized jumps. The long-term **DRM** (capacity C_D) is the curated recovery bank; an embedding is promoted only after proving *temporally stable* — at least $m_{\min}=3$ of the last $W=10$ RAM embeddings agree with it above $\tau_{\text{sim}}=0.75$:

$$\sum_{j=t-W+1}^t \mathbf{1}[\cos(\psi_j, \psi_t) \geq \tau_{\text{sim}}] \geq m_{\min}. \quad (9)$$

Transient views under blur or partial occlusion never gather enough agreeing neighbours, so every DRM anchor is an appearance the target genuinely held for a while, not a one-frame fluke.

7 Multi-Phase Reacquisition



Figure 4: Recovery decouples three questions: the EKF supplies *where*, the DRM bank ranks *which*, and SiamABC *confirms* before control is returned.

Declaring loss and a fast path. When $\hat{c} < \tau=0.50$, occlusion is declared, the box is frozen, and the EKF runs predict-only. Patience is deliberately short (one sub- τ frame): recovery is cheap and self-verifying, so a false alarm costs a single detector pass before the verifier waves the tracker back. We first try a fast path — re-run SiamABC at the EKF-predicted location; a strong, identity-consistent return ($\hat{c} \geq 0.80$, appearance ≥ 0.60) resumes immediately. Otherwise we enter three phases.

Phase 1 — *where* (YOLO). A YOLOv11n detector proposes boxes in a square ROI centred on the EKF prediction, whose side grows with the lost-frame count n ,

$$r_t = \min(r_0 g^{\lfloor n/T_g \rfloor}, r_{\max}), \quad (10)$$

starting wide ($r_0=20\times$) and widening geometrically ($g=1.4$ every $T_g=150$ frames), with a tighter schedule ($r_0=8\times$, $r_{\max}=40\times$) for tiny targets ($< 3\%$ of frame) so their pixels survive. YOLO runs at 320×320 and up to five detections are embedded.

Phase 2 — *which* (DRM). Each candidate c is scored by its best match to the bank,

$$S(c) = \max_k \cos(\psi_k, \phi(I_t, c)), \quad (11)$$

and the top $K=3$ advance. This is the primary distractor defence: a look-alike in the right place still scores low, matching the target’s *position* but not its *identity*.

Self-calibrating acceptance margin. A fixed acceptance threshold m does not transfer across videos — a textured target self-scores high, a distant one low. During normal tracking we periodically measure the genuine target’s self-score S^* and track its moving average, setting the margin just below it:

$$\bar{S}^* \leftarrow (1-\alpha_m)\bar{S}^* + \alpha_m S^*, \quad m = \text{clip}(\bar{S}^* - \delta, \tau_{\min}, \tau_{\max}), \quad (12)$$

with $\alpha_m=0.2$, $\delta=0.05$, $[\tau_{\min}, \tau_{\max}]=[0.50, 0.80]$ (warm-started at 0.70 until five measurements accrue) — tight enough to reject distractors, loose enough to admit the true reacquisition, with no per-video knob.

Phase 3 — *confirm*. The detector does not get the final word. The best candidate, advanced one EKF-velocity step, re-initialises SiamABC, and recovery ends only if the tracker agrees ($\hat{c} \geq 0.70$); otherwise it retries. Using the resuming tracker as its own verifier prevents one confident false detection from hijacking the identity.

8 Efficiency and Real-Time Deployment

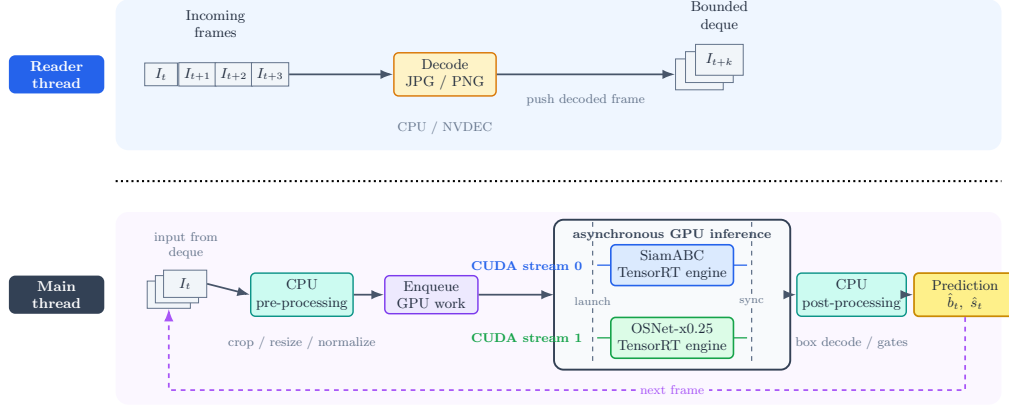


Figure 6: Inference path across the reader and main threads. **Stream 0** runs the SiamABC TensorRT engine; **Stream 1** runs OSNet in parallel. OSNet’s descriptor for frame N is computed while SiamABC tracks frame $N+1$, so its cost hides behind the tracker.

Component	Params	Compute	Export
SiamABC backbone (FP16)	~ 9.0 M	~ 5.0 GFLOPs	TRT FP16
SiamABC head (FP32)	~ 0.20 M	< 0.1 GFLOPs	TRT FP32
OSNet-x0.25	0.20 M	0.16 GFLOPs	TRT FP16
YOLOv11n @ 320	2.6 M	~ 1.6 GFLOPs	ONNX-TRT FP16

Table 1: Per-component budget. Only the backbone and head run every normal frame; YOLO and OSNet run only during recovery.

Efficiency is the evidence we *can* report now: the system runs in real time on a Jetson Orin Nano because of three decisions — a small model stack, per-layer-precision compilation, and an I/O pipeline that never stalls the GPU.

7.1 Precision split: FP16 backbone, FP32 head

Running everything in FP16 for the free $2\times$ is right for most of the network and wrong for one piece. The **backbone and necks** (E, N, N') hold nearly all FLOPs but emit intermediate features no decision reads directly; FP16 rounding averages out before any threshold, so they compile to a TensorRT FP16 engine. The **head** T is different: its output *is* a decision. $\hat{c} = \sigma(\ell)$ meets hard thresholds (0.50 to enter occlusion, 0.70 to confirm), and FP16 carries only 10 mantissa bits, so near a unit-scale logit its values are spaced $\approx 2^{-10} \approx 10^{-3}$ apart — a genuine 0.501 can round below 0.50 and trigger a phantom occlusion every frame. The head also holds the TTBN layers (Eq. 5), whose running-statistics blend degrades in FP16. Hence the rule: *the head stays FP32*, at < 0.1 GFLOPs a negligible cost.

Backbone modes and tracing. We ship `dynamic_fp16` (default): one engine spans both crop sizes under a single profile `[min=128,opt=128,max=256]`; `static_fp16`, `static_fp16_fp32_neck`, and `dynamic_fp32` are alternates. The engine is built from `torch.jit.trace`, not `torch.export`: the export guard pass hits the backbone’s stride-2 divisibility constraint $\text{Mod}((s-1)/2, 2) \neq 0$ for some $s \in [128, 256]$ and raises `ConstraintViolationError`, whereas tracing records ops on a dummy input and never inspects shape guards, so one dynamic profile covers both sizes.

Auxiliary engines. The attention neck uses `torch.compile` where Triton exists (x86/CUDA), else a per-size static TRT engine on Jetson (ARM64), validated against eager to 10^{-2} . **OSNet** is compiled to a static batch-1 FP16 TensorRT engine at 256×128 (batch-1 because its post-pool `view(v.size(0), -1)` breaks TRT’s dynamic-batch converter), so recovery descriptors are accelerated rather than left in eager mode. **YOLOv11n** exports ONNX $\rightarrow$ TRT FP16 at 320×320 ; halving resolution from 640 cuts cost $\approx 4\times$ ($6.5 \rightarrow 1.6$ GFLOPs), affordable because the EKF ROI already localises it.

Configuration	FPS	ms	Readout
Full pipeline	65.9	15.2	Baseline for relative ablations.
– without TensorRT	26.6	37.5	$2.5\times$ slower; precision-split TRT is the dominant real-time factor.
– without reader thread	59.7	16.7	$\approx 10\%$ loss; hiding decode behind compute earns its place.
Jetson Orin Nano	32	31.2	Real-time class with software decode; GStreamer/NVDEC should improve the remaining decode margin.

Table 2: FPS ablation over 15 UAV sequences (2082 frames). RTX 3050 Laptop GPU rows isolate relative optimisation gains; the Jetson row reports deployment-target behaviour.

Per-mode budget. Gating turns the small stack (Table 1) into difficulty-tracking cost. **Normal tracking** (≈ 5.4 GFLOPs/frame): both template branches are cached and re-encoded only when $\hat{z}$ rotates (every $N=40$ frames), so steady-state cost is one search encode plus the tiny head. **Recovery** (≈ 2.4 GFLOPs/frame, avg): SiamABC does *not* run every recovery frame — a typical frame pays one YOLO call (≈ 1.6 GFLOPs) plus OSNet on a few crops (≤ 0.8 GFLOPs), the full 5.4 GFLOPs pass only on a confirming frame. The expensive state is thus, on average, *cheaper* than normal tracking.

Hiding I/O latency. A **reader thread** decodes frames (via Jetson NVDEC when available) into a bounded queue while the **main thread** launches GPU work, so frame $N+1$ decodes while the GPU tracks frame N — decode hides behind compute, byte-identically. SiamABC and OSNet share no within-frame data (SiamABC takes the current crop, OSNet the previous box), so they run on two CUDA streams (Fig. 6) concurrently *when both are active*, dropping per-frame GPU time from a sum to a maximum on the recovery frames where OSNet is busy,

$$t_{\text{frame}} \approx \max(t_{\text{SiamABC}}, t_{\text{OSNet}}) + t_{\text{serial}}. \quad (13)$$

TRT engines are cached as TorchScript `.ts` keyed by weights, GPU, and versions (first build 1–3 min, reload < 1 s); the engine cache is visible in Table 2, where building the TRT engine once lets the remaining runs reload it in seconds rather than rebuild.

7.2 Measured throughput ablation

The per-mode budget above is a FLOP count; we also measured wall-clock throughput to confirm each deployment decision earns its place. Table 2 reports the FPS ablation over 15 UAV sequences (2082 frames) on an RTX 3050 Laptop GPU. The numbers therefore give a *relative* attribution of the retained deployment optimisations, not the absolute on-device rate of the Jetson Orin Nano deployment target (§8).

8 Limitations and Scope

This paper presents the system design and its measured compute budget; it does not yet report tracking-accuracy numbers, and we state that boundary plainly. The robustness claims (recalibration separating target from non-target, the memory rejecting distractors, the camera-aware EKF surviving pans) are argued mechanistically and from the deployed configuration, not from a benchmark. Their empirical validation — success and precision on UAV123, UAVTrack112, and DTB70 (the fine-tuning domains), with per-module ablations and a calibration-separation curve for $\hat{c}$ — is the immediate next step and the natural home for the claim in §4 that recalibration opens a usable margin. Two scope notes follow from the deployed parity configuration: the candidate-ranking score is appearance-only (its motion and geometry terms are disabled), so distractor rejection rests on identity rather than a fused score; and the reported thresholds are conservative competition settings rather than per-benchmark optima. Finally, the throughput ablation (Table 2) was measured on an RTX 3050 Laptop GPU and attributes each optimisation’s *relative* contribution; the on-device Jetson Orin Nano rate we report was logged with software frame decode (hardware NVDEC/GStreamer disabled) over a sample of sequences, so it is a conservative figure that the hardware-decode path is expected to raise.

9 Conclusion

SiamRAM makes a fast Siamese core trustworthy by recalibrating its confidence, then gates three robustness modules on that confidence so they engage only when needed: a camera-aware filter for *where*, an identity memory for *which*, and the tracker itself to *confirm*. A precision-split TensorRT pipeline and an asynchronous reader–inference architecture keep the whole system within a real-time embedded budget. Quantitative accuracy evaluation, scoped in §8, is the next step.

References

- [1] L. Bertinetto et al., “Fully-Convolutional Siamese Networks for Object Tracking,” *ECCV Workshops*, 2016.
- [2] B. Li et al., “SiamRPN++: Evolution of Siamese Visual Tracking with Very Deep Networks,” *CVPR*, 2019.
- [3] “SiamABC: Siamese Tracking with Adaptive Batch Calibration,” *arXiv*, 2025.
- [4] K. Zhou et al., “Omni-Scale Feature Learning for Person Re-Identification,” *ICCV*, 2019.
- [5] “MATA: Motion-Aware Test-Time Adaptation for Visual Tracking,” *arXiv*, 2025.
- [6] Z. Tian et al., “FCOS: Fully Convolutional One-Stage Object Detection,” *ICCV*, 2019.
- [7] T.-Y. Lin et al., “Focal Loss for Dense Object Detection,” *ICCV*, 2017.
- [8] B. D. Lucas and T. Kanade, “An iterative image registration technique with an application to stereo vision,” *IJCAI*, pp. 674–679, 1981.
- [9] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *J. Basic Engineering*, vol. 82, pp. 35–45, 1960.